

Model-Driven Software Development

Xtext and Code Generator



Miguel Campusano

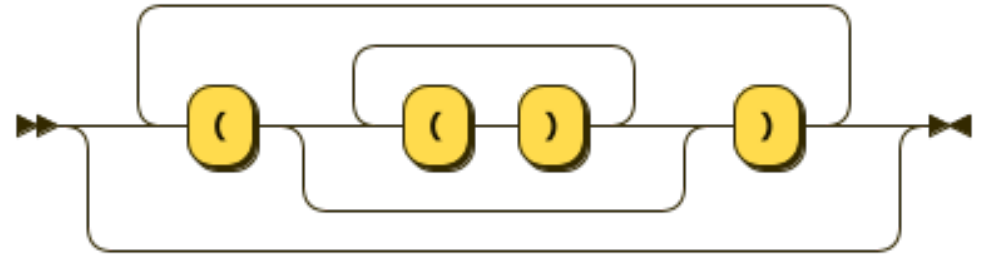
SDU Software Engineering

Previous class

External DSL Background

Summary

Entity example syntax definition
From External DSL to Metamodel
From *Program* to Metamodel
From *Text* to Metamodel
XText warm-up



Xtext

Text to Metamodel instance

Lexing

String to Tokens (Strings with meaning)

Parsing

Tokens to Data Structure (Usually an Abstract Syntax Tree)

Name resolution

Identifiers associated to other elements

Function Call *resolves* to a Function Definition

Variable Usage *resolves* to Variable Definition

...

Context-Free Grammar: BNF

Backus-Naur Form

Notation to define context-free grammars

DSL for specifying programming languages

BNF for balanced parentheses?

Context-Free Grammar: EBNF

Extended BNF

BNF with quality-of-life symbols

It is not more *powerful*

Anything you express in EBNF can be expressed using BNF

It is more convenient

Extensions

* Kleene star: 0 or more occurrences

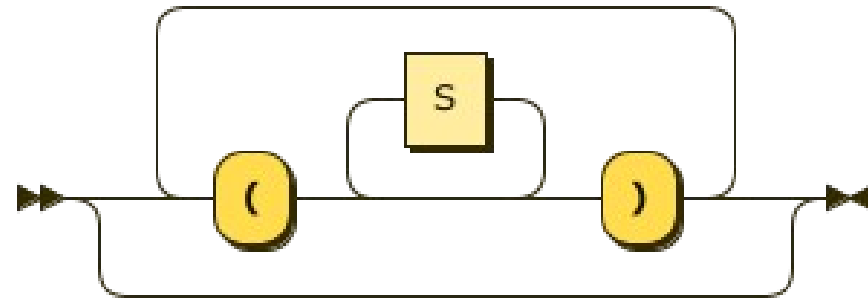
+ Kleene plus: 1 or more occurrences

? Optional: 0 or 1 occurrences

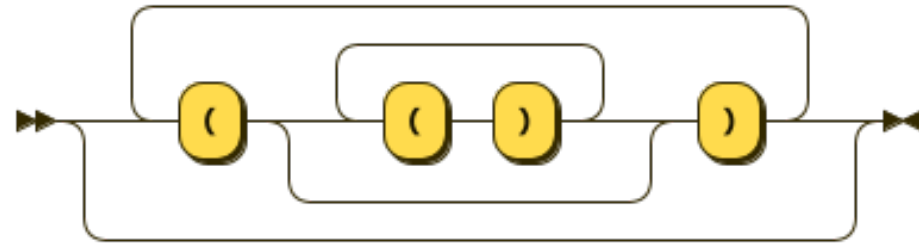
EBNF for balanced parenthesis?

EBNF for balanced parentheses

$S ::= (' (' S^* ') ')^*$



Source: <https://www.bottlecaps.de/rr/ui>



Text to Metamodel instance

Lexing

String to Tokens (Strings with meaning)

Parsing

Tokens to Data Structure (Usually an Abstract Syntax Tree)

Name resolution

Identifiers associated to other elements

For example: Function Call *resolves* to a Function Definition

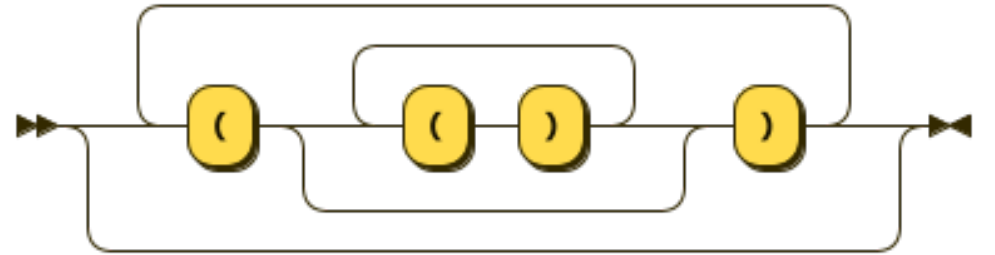
The logo for Xtext, featuring the word "Xtext" in a stylized, bold, sans-serif font. The "X" is large and prominent, followed by "t", "e", and "x", and then "t". The "x" is stylized with a curved, almost calligraphic touch.

Group Project Work

Week 4

EBNF specification

Xtext Grammar



Xtext

Discussion

Discussion: Code generation

Your project

You have a DSL (EBNF, syntax)

What is the next step? Generate *something*!

What do you generate?

What is the output of your program?

What is the *execution* of your program?

What properties may have the generated text/code?

eg., should be readable? Modular? Scalable? ...

Model-Driven Software Development

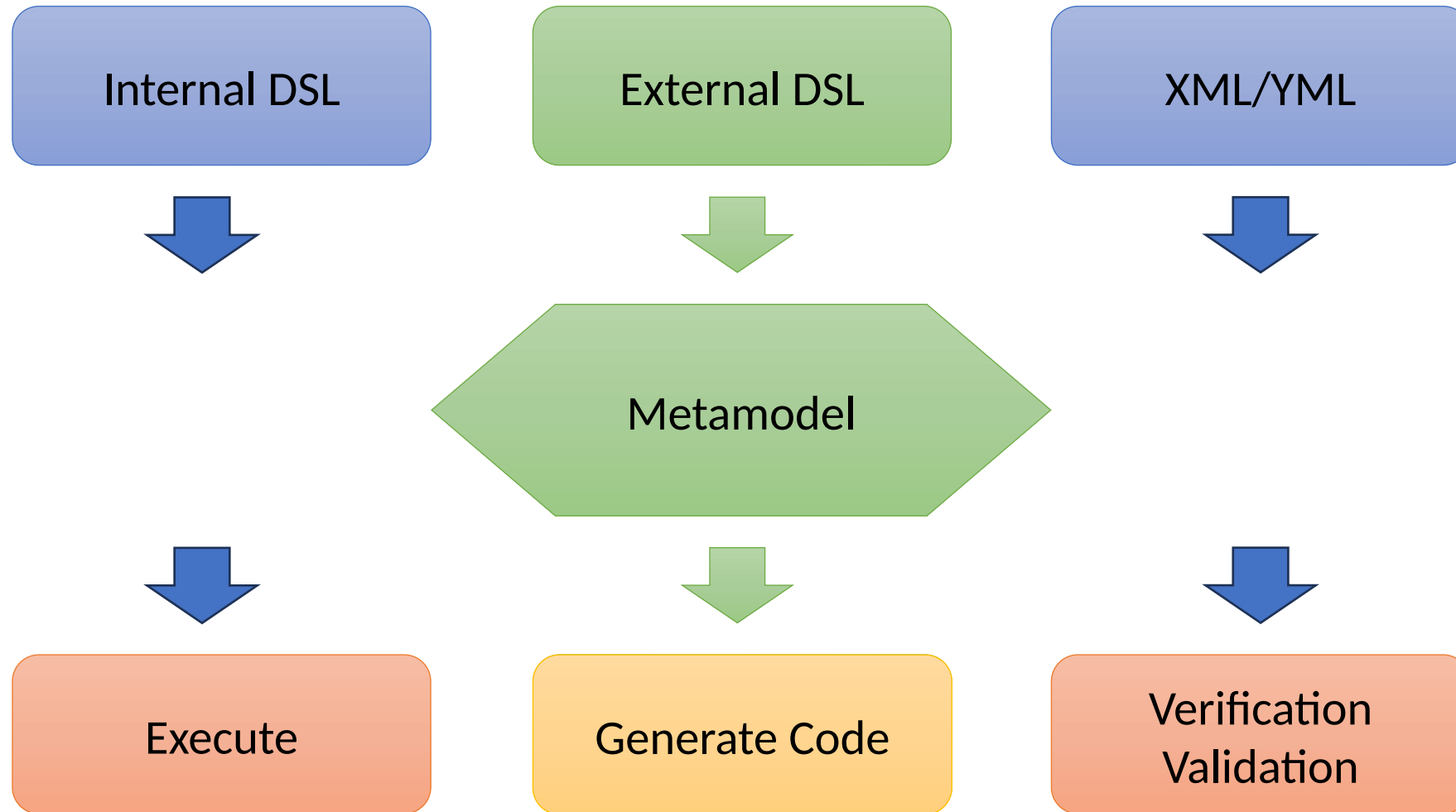
Xtext and Code Generator



Miguel Campusano

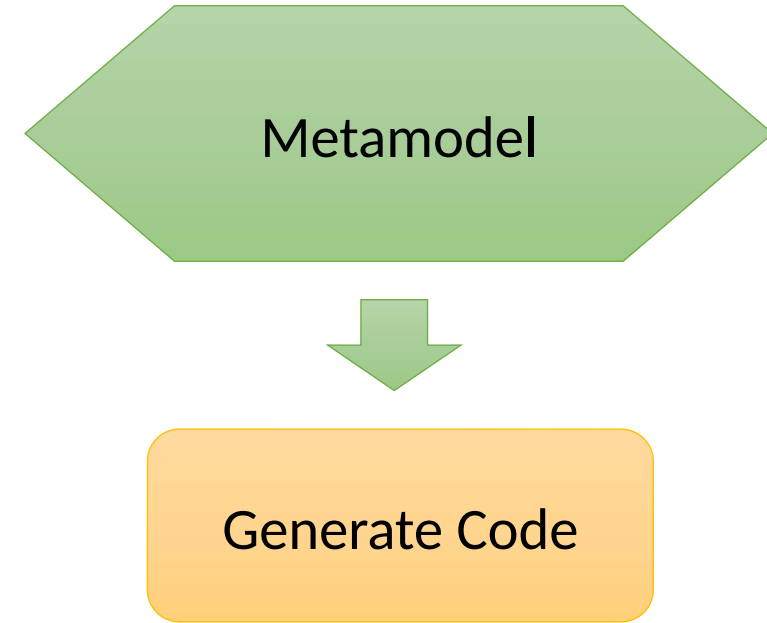
SDU Software Engineering

Metamodel: Internal DSL interpretation



Today

Xtext grammar
Code generation
Xtext - Xtend



Xtext

Xtext

Xtext EBNF

Lexing, Parsing and Name Resolution for *free*

It generates necessary code to parse your DSL and *more*

Xtext: Entity

```
... //prelude
SystemRoot: 'system' ID Entity* ... ;
Entity: 'entity' //keyword (purple color in IDE)
        ID //ID: non terminal rule for ids
        Attribute* // 0 or more Attribute
;
Attribute: ...
...
```

Code Generation

Code Generation

Technically a compiler: translates language L1 into language L2

L1 is our source language, often defined using a context-free grammar

L2 is our target language, could be a binary language, but also *source*

Phases: lexing - parsing - name resolution - analysis - code generation

Code Generation

We say code generation, but you can do other things:

- Interpret code

- Verify/Validate properties

- Generate *text/code* to be used in other programs/systems

 - Source code to be executed

 - XML automata representation for UPPAAL (verification/validation)

 - Documentation / Visualizations

 - ...

Code Generation

In this course: DSL that generates source code

Internal/External DSL creates a metamodel instance

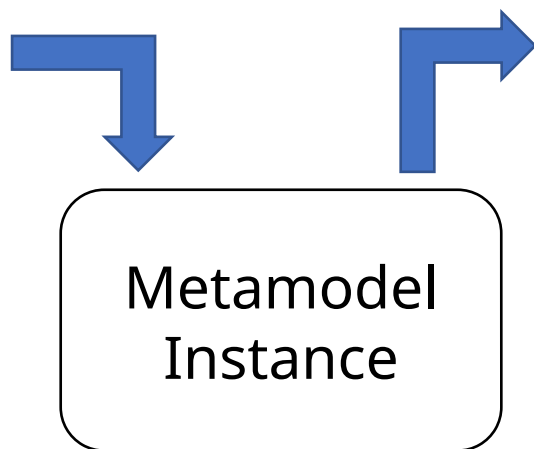
From metamodel instance, code is generated

Code can be executed by an external agent

External DSL: Xtext (syntax) and *Xtend* (code generation)

Code Generation: idea

```
1 system University
2
3 entity Person
4     attribute name is String
5     attribute age is Number
6
7 entity Student
8     - id is Number
9
10 entity Teacher
11
12 entity Course
13     attribute title is String
14
15 Student is Person
16 Teacher -> Person
17 Teacher 1--1 Course
18 Student *- - * Course
```



```
package university;

import java.util.ArrayList;

public class Student extends Person {

    private int id;
    private List<Course> courses = new ArrayList<>();

    public Student(int id, String name, int age) {
        super(name, age);
        this.setId(id);
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public List<Course> courses() {
        return courses;
    }

    public void addCourse(Course course) {
        courses.add(course);
    }

}
```

Code Generation: approach

Metamodel
Instance



```
for each Entity e in system.entities:
  make new file("e.name.java")
  add "package system.name;"
  add "java.util.*;"
  add "public class e.name {" //extend?
  for each Attribute a in e.attributes:
    add "private a.type a.name;" //type?
  . . .
```

```
package university;

import java.util.ArrayList;

public class Student extends Person {

    private int id;
    private List<Course> courses = new ArrayList<>();

    public Student(int id, String name, int age) {
        super(name, age);
        this.setId(id);
    }

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public List<Course> courses() {
        return courses;
    }

    public void addCourse(Course course) {
        courses.add(course);
    }

}
```

Code Generation: Java

Metamodel
Instance



```
for(Entity e: system.getEntities()) {  
    PrintWriter f = make_new_file(e.getName()+  
        ".java");  
    f.println("package "+system.name);  
    f.println("public class "+e.getName()+"{");  
    for(Attribute a: e.attributes) {  
        f.println("    private "+typeFrom(a.getType()))  
        . . .  
    }
```

```
package university;  
  
import java.util.ArrayList;  
  
public class Student extends Person {  
  
    private int id;  
    private List<Course> courses = new ArrayList<>();  
  
    public Student(int id, String name, int age) {  
        super(name, age);  
        this.setId(id);  
    }  
    public int getId() {  
        return id;  
    }  
    public void setId(int id) {  
        this.id = id;  
    }  
  
    public List<Course> courses() {  
        return courses;  
    }  
  
    public void addCourse(Course course) {  
        courses.add(course);  
    }  
}
```


Code Generation: Xtend

Metamodel
Instance

```
for(Entity e: system.getEntities()) {  
    generateFile(e.name+".java",  
        '''package university;  
import java.util.*  
public class «e.name» {  
    «FOR a: e.attributes»  
    private «a.type.javaType» «a.name»;  
    . . .  
    '''  
}
```



```
package university;  
import java.util.ArrayList;  
public class Student extends Person {  
    private int id;  
    private List<Course> courses = new ArrayList<>();  
    public Student(int id, String name, int age) {  
        super(name, age);  
        this.setId(id);  
    }  
    public int getId() {  
        return id;  
    }  
    public void setId(int id) {  
        this.id = id;  
    }  
    public List<Course> courses() {  
        return courses;  
    }  
    public void addCourse(Course course) {  
        courses.add(course);  
    }  
}
```

Xtend

Xtend

Java dialect that facilitates program generation

Template-base: previous slide

Cleaner: Type inference, functional style (concise lambdas), etc...

Extension methods

Type-case: dispatch annotation and type switch

Xtend: cleaner

Type inference

```
def display(Model model, OutputStream stream) { ... }  
public void display(...){ ... }
```

Type inference: not always

```
def abs(int x) { if(x<0) -x else x }  
def int fac(int x){ if (x==0) 1 else x*fac(x-1) }
```

Lambdas

```
entity.attributes.map[name] //or use [it.name]
```

Xtend: extension methods

```
def display(Model model, OutStream stream) { ... }
```

```
...
```

```
model.display(System::out)
```

```
display(model, System::out)
```

Xtend: type-case and pattern matching

```
def dispatch display(Attribute a){ ... }  
def dispatch display(Relation r){ ... }  
...  
model.dispatch
```

```
switch model {  
  Attribute: ...  
  Relation: ...  
}
```

Xtend: what about method overload?

```
String prettyPrint(Integer x) { return "int: "+x; }  
String prettyPrint(String x) { return "<"+x+">"; }  
String prettyPrint(Object x) { return "["+x+"]"; }
```

```
Integer i = 5;  
String s1 = prettyPrint(i);  
System.out.println(s1); // L1  
Object o = i;  
String s2 = prettyPrint(o);  
System.out.println(s2); // L2
```

```
def dispatch prettyPrint(Integer x) { return "int: "+x; }  
def dispatch prettyPrint(String x) { return "<"+x+">"; }  
def dispatch prettyPrint(Object x) { return "["+x+"]"; }
```

```
val Integer i = 5;  
s1 = prettyPrint(i);  
System::out.println(s1); // L3  
val Object o = i;  
s2 = prettyPrint(o);  
System::out.println(s2); // L4
```

Xtext & Xtend

Xtext: Entity

```
... //prelude
SystemRoot: Entity* ... ;
Entity: 'entity' //keyword (purple color in IDE)
        ID //ID ...
        Attribute* // 0 or more non-terminals
;
Attribute: ...
...
```

Use metamodel in xtend

Xtext: we parse valid programs

How do we use the information of programs?

Eg., **entity** person ...

How do we get the name of the entity?

Xtext: Attribute generation

```
... //prelude
SystemRoot: Entity* ... ;
Entity: 'entity' //keyword (purple color in IDE)
        name=ID //ID ...
        attributes+=Attribute* // 0 or more non-terminals
;
Attribute: ...
...
```

Assignment (=): create *attribute* with that value

Addition (+=): create array of elements on *attribute*

Xtext: Class generation

```
... //prelude
SystemRoot: Entity* ... ;
Entity: 'entity' //keyword (purple color in IDE)
        name=ID //ID ...
        attributes+=Attribute* // 0 or more non-terminals
;
Attribute: ...
...
```

Rule: Create class (interface) of same name

Xtext: Name resolution

```
... //prelude
SystemRoot: Entity* ... ;
Entity: 'entity' //keyword (purple color in IDE)
        name=ID //ID ...
        attributes+=Attribute* // 0 or more non-terminals
;
Inheritance: classA=[Entity] ('is'|'->') classB=[Entity];
```

Assignment + square brackets:

Assignment as before, but use ID rule to match the name of [*Class*]

Xtend

Live demo: starting point

```
system University

entity Person
    attribute name as string
    attribute age as number

entity Student
    attribute id as number

entity Teacher

entity Course
    attribute title as string

relation Student is Person
relation Teacher is Person
relation Teacher have Course
relation many Student has many Course
```

In-class exercise: code generation

Smallest program that you can have: parse & generate

System, one entity with name, no attributes, no associations

Add to discussion board

Next?

Attributes (0 or more)

More entities (0 or more)

Inheritance

One-to-one, one-to-many, many-to-one, many-to-many

...

Implementation notes

- Write grammar, refine, check metamodel, etc

 - For grammar: use 2 or 3 examples

- Use existing code as a starting point

 - Insert into template

 - Incrementally add Xtext annotations

- Overall structure: template approach

- Specific code: extract from metamodel (transformer)

- Keep templates as simple as possible (use helper functions)

- Exploit Xtend features (type-based switch, etc...)

Summary

Summary

Xtext grammar
Code generation
Xtext - Xtend

Next:
Refine Class Description DSL

```
for(Entity e: system.getEntities()) {  
    generateFile(e.name+".java",  
        '''package university;  
import java.util.*  
public class «e.name» {  
    «FOR a: e.attributes»  
    private «a.type.javaType» «a.name»;  
    . . .  
    '''  
}
```

The logo for Xtend, featuring a stylized blue 'X' followed by the word 'tend' in a dark grey sans-serif font.

Group Project Work

Week 5

Example programs:

DSL

Output program

Code generator in Xtend

```
for(Entity e: system.getEntities()) {  
    generateFile(e.name+".java",  
        '''package university;  
import java.util.*  
public class «e.name» {  
    «FOR a: e.attributes»  
    private «a.type.javaType» «a.name»;  
    . . .  
    '''  
}
```



Next

Abstract Syntax Tree

Advance Parsing Topics